

These edge variable values are replaced by integer `x` and `y` coordinates in the code that comes next:

Python

```
for x in range(baseImageWidth):
    for y in range(baseImageHeight):
        if baseImage.getpixel((x, y)) == magentaColor:
            if magentaLeft is None or x < magentaLeft:
                magentaLeft = x
            if magentaRight is None or x > magentaRight:
                magentaRight = x
            if magentaTop is None or y < magentaTop:
                magentaTop = y
            if magentaBottom is None or y > magentaBottom:
                magentaBottom = y
```

These nested `for` loops iterate the `x` and `y` variables over every possible `x`, `y` coordinate in the base image. We check whether the pixel at each coordinate is the pure magenta color stored in `magentaColor`, then update the `magentaLeft` variable if the coordinates of the magenta pixel are further left than currently recorded in `magentaLeft`, and so on for the other three directions.

By the time the nested `for` loops are finished, `magentaLeft`, `magentaRight`, `magentaTop`, and `magentaBottom` will describe the boundaries of the magenta pixels in the base image. If the image has no magenta pixels, these variables will remain set to their initial `None` value:

Python

```
if magentaLeft is None:
    # BASE CASE - No magenta pixels are in the image.
    return baseImage
```

If `magentaLeft` (or really, any of the four variables) is still set to `None` after the nested `for` loops complete, no magenta pixels are in the image. This is a base case for our recursive algorithm because the magenta area becomes smaller and smaller with each recursive call to `makeDroste()`. At this point, the function returns the Pillow `Image` object in `baseImage`.